

Sprechtext zur Präsentation State Pattern mit Spring Boot

Retro Kit Order Workflow

16. Mai 2026

Folie 1: Titel

Heute präsentieren wir unser Spring-Boot-Projekt „Retro Kit Order Workflow“. Das Thema unserer Arbeit ist das State Pattern an einem Bestellsystem. Dabei vergleichen wir bewusst zwei Branches: eine Ausgangslösung ohne gezielten Pattern-Einsatz und eine refaktorierte Lösung mit State Pattern.

Folie 2: Ausgangsproblem

Eine Bestellung durchläuft mehrere fachliche Zustände. Typische Beispiele sind erstellt, bezahlt, verpackt, versendet oder storniert. Wenn man diese Logik direkt in einer Service-Klasse umsetzt, entstehen schnell viele Fallunterscheidungen. Genau das ist unser Problem im Branch **without-pattern**: Die Zustandslogik liegt zentral im `KitOrderService` und wird dadurch mit jeder neuen Anforderung unübersichtlicher.

Folie 3: Projektüberblick

Unser Projekt ist ein eigenständiges Maven- und Spring-Boot-Projekt mit Java 21, Spring Web, Spring Data JPA und einer H2-Datenbank. Fachlich geht es um Retro-Fußballtrikot-Bestellungen. Besonders wichtig ist die Branch-Struktur: Im Branch **without-pattern** liegt die funktionierende Ausgangslösung, im Branch **with-pattern** die refaktorierte Lösung mit klarer Pattern-Struktur.

Folie 4: Erklärung des State Patterns

Das State Pattern bedeutet, dass das Verhalten eines Objekts vom aktuellen Zustand abhängt und dieser Zustand nicht mit großen `if`-Blöcken, sondern mit eigenen Klassen modelliert wird. Statt dass eine einzige Service-Klasse alle Regeln kennt, kapselt jede Zustandsklasse die erlaubten Übergänge für genau einen Status.

Folie 5: Fachlicher Bestellablauf

Unsere Bestellung startet im Zustand `CREATED`. Danach kann sie bezahlt werden, also in `PAID` wechseln. Danach folgt `PACKED`, dann `SHIPPED`. Zusätzlich kann eine Bestellung in mehreren frühen Phasen storniert werden. Als sinnvolle Erweiterung haben wir `RETURNED` eingebaut. Gerade dieser zusätzliche Zustand zeigt sehr gut, warum das State Pattern die Erweiterbarkeit verbessert.

Folie 6: without-pattern

Im **without-pattern**-Branch steckt die Logik noch zentral im `KitOrderService`. Dort wird direkt geprüft, ob der aktuelle Status passt. Zum Beispiel darf man nur eine **CREATED**-Bestellung bezahlen oder nur eine **PACKED**-Bestellung versenden. Das funktioniert zwar, aber die Klasse wird schnell groß, stark gekoppelt und schwerer wartbar.

Folie 7: with-pattern

Im **with-pattern**-Branch haben wir die Zustandslogik in einzelne Klassen ausgelagert. Es gibt ein gemeinsames Interface `OrderState`, eine `OrderStateFactory` und konkrete Klassen wie `CreatedOrderState`, `PaidOrderState` oder `ShippedOrderState`. Der `KitOrderService` orchestriert dann nur noch und delegiert die eigentliche Regel an den passenden State.

Folie 8: REST-Endpunkte

Die Anwendung erfüllt die Anforderungen des Arbeitsblatts deutlich. Es gibt mehr als drei REST-Endpunkte. Besonders wichtig sind die Endpunkte zum Anlegen der Bestellung, zum Anzeigen nach ID oder Status und zu den Statuswechseln wie Bezahlen, Verpacken, Versenden, Stornieren und Retournieren.

Folie 9: Tests und Qualität

Wir haben sowohl Integrations- als auch Unit-Tests verwendet. Die Integrationstests prüfen, ob die REST-API und die Statuswechsel korrekt funktionieren. Im **with-pattern**-Branch kommen zusätzlich gezielte Unit-Tests für die State-Klassen dazu. Das ist ein Vorteil des Patterns, weil die Fachregeln isolierter getestet werden können.

Folie 10: Vergleich der Branches

Der wichtigste Unterschied ist die Verantwortungstrennung. Ohne Pattern sammelt eine zentrale Service-Klasse immer mehr Wissen über alle Zustände. Mit Pattern wandert dieses Wissen in die jeweiligen Zustandsklassen. Das macht die Lösung klarer, testbarer und vor allem leichter erweiterbar. Der Nachteil ist, dass mehr Klassen entstehen und die Struktur zunächst etwas aufwendiger wirkt.

Folie 11: Rolle der KI

Generative KI hat uns vor allem in drei Punkten geholfen: beim Verstehen des State Patterns, beim Erstellen eines Refactoring-Plans und beim Ableiten sinnvoller Testfälle. Gleichzeitig mussten wir die Vorschläge prüfen, weil manche Antworten zu allgemein oder für Spring Boot nicht konkret genug waren. Die endgültigen fachlichen Entscheidungen haben wir selbst getroffen.

Folie 12: Demo-Ablauf

In der Live-Demo würden wir zuerst den **without-pattern**-Branch zeigen und kurz erklären, wo die Fallunterscheidungen liegen. Danach wechseln wir auf **with-pattern**, zeigen die State-Klassen und demonstrieren einen kompletten Ablauf von **CREATED** bis **SHIPPED** oder **RETURNED**.

Folie 13: Fazit

Unser Fazit ist: Das State Pattern lohnt sich überall dort, wo Objekte einen klaren Lebenszyklus mit zustandsabhängigem Verhalten besitzen. Für dieses Bestellsystem ist es gut geeignet, weil die Bestellung nicht nur Daten speichert, sondern sich je nach Zustand fachlich unterschiedlich verhält. Für sehr kleine Systeme wäre der zusätzliche Strukturaufwand möglicherweise noch nicht nötig.